Linear Discriminant Analysis (LDA)

1. The Intuition: Generative Modeling

LDA is a classification method that assumes data comes from Gaussian distributions.

Generative Model Definition:
We assume the data X for each class $Y = y$ is generated from a Normal (Gaussian) distribution:

$$X \mid Y = y \sim \mathcal{N}(\mu_y, \Sigma)$$

Crucially, LDA assumes all classes share the **same** covariance matrix Σ (same shape/spread), but have different means μ_y.

Instead of drawing a random line, LDA calculates the probability of a new point belonging to each class.

The classifier assigns the label $\hat{y}$ that maximizes this probability:

$$\hat{y} = \arg\max_y \ \log \pi_y - \tfrac{1}{2}(x - \mu_y)^\top \Sigma^{-1}(x - \mu_y)$$

Working Example: Classifying a New Point

Suppose we have two classes (Red and Blue) with:

- **Means:** $\mu_{red} = [2, 2]$, $\mu_{blue} = [6, 6]$
- **Shared Covariance:** $\Sigma = I$ (Identity matrix for simplicity)
- **Priors:** Equal probability $\pi_{red} = \pi_{blue} = 0.5$ (so $\log \pi$ cancels out)

New Point: We want to classify $x = [3, 3]$.

Step 1: Calculate Distance to Red Mean

$$d_{red} = (x - \mu_{red})^\top \Sigma^{-1} (x - \mu_{red})$$

$$= ([3,3] - [2,2])^\top I ([3,3] - [2,2])$$

$$= [1,1]^\top [1,1] = 1^2 + 1^2 = \mathbf{2}$$

Step 2: Calculate Distance to Blue Mean

$$d_{blue} = (x - \mu_{blue})^\top \Sigma^{-1} (x - \mu_{blue})$$

$$= ([3,3] - [6,6])^\top I ([3,3] - [6,6])$$

$$= [-3,-3]^\top [-3,-3] = (-3)^2 + (-3)^2 = \mathbf{18}$$

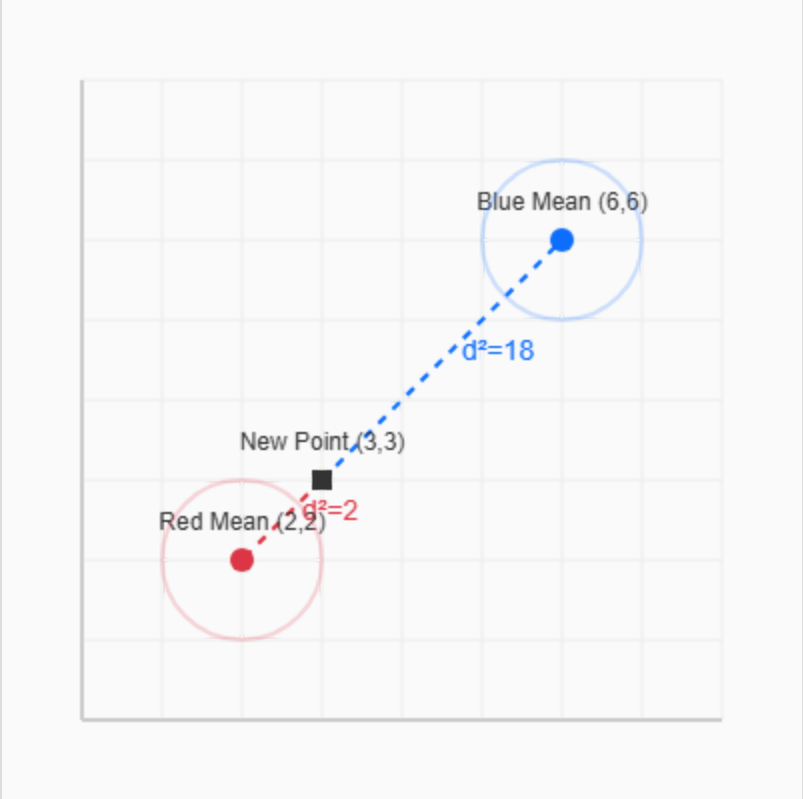
Step 3: Compare Scores

Since we want to *maximize* the term with the negative distance (or *minimize* the distance):

$$\text{Score}_{red} \propto -2 \quad > \quad \text{Score}_{blue} \propto -18$$

Result: The point is closer to the Red mean (Distance 2 < 18), so we classify it as **Red**.

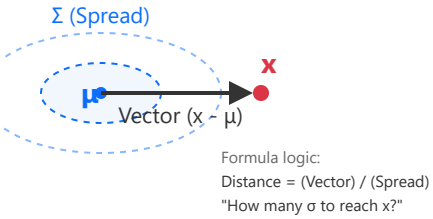
Visualizing the Example



Intuitively, the classification score depends on two key factors:

1. Mahalanobis Distance

"Distance normalized by spread"

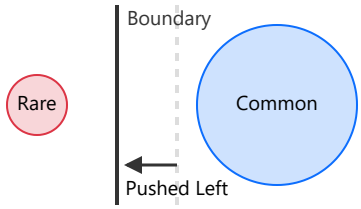


Term:

$$-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)$$

2. Prior Probability (π_y)

"Baseline popularity of the class"



If **Blue** is very common, the boundary shifts to give it more space.

Term:

$$+\log(\pi_y)$$

Learning Classification Boundary

In LDA, we assume that the prior probabilities π_y, the mean vectors $\boldsymbol{\mu}_y$, and the covariance matrix Σ are all unknown.

To estimate these from the data, we use:

$$\widehat{\pi}_y = \frac{|\{i : y_i = y\}|}{n}$$

$$\widehat{\boldsymbol{\mu}}_y = \frac{1}{|\{i : y_i = y\}|} \sum_{i:y_i=y} \mathbf{x}_i$$

$$\widehat{\Sigma} = \frac{1}{n} \sum_{y=0}^{K-1} \sum_{i:y_i=y} (\mathbf{x}_i - \widehat{\boldsymbol{\mu}}_y)(\mathbf{x}_i - \widehat{\boldsymbol{\mu}}_y)^T$$

$$\mathbf{w} = \widehat{\Sigma}^{-1}(\widehat{\boldsymbol{\mu}}_1 - \widehat{\boldsymbol{\mu}}_0)$$

$$b = \frac{1}{2}\widehat{\boldsymbol{\mu}}_0^T \widehat{\Sigma}^{-1} \widehat{\boldsymbol{\mu}}_0 - \frac{1}{2}\widehat{\boldsymbol{\mu}}_1^T \widehat{\Sigma}^{-1} \widehat{\boldsymbol{\mu}}_1 + \log \frac{\widehat{\pi}_1}{\widehat{\pi}_0}$$

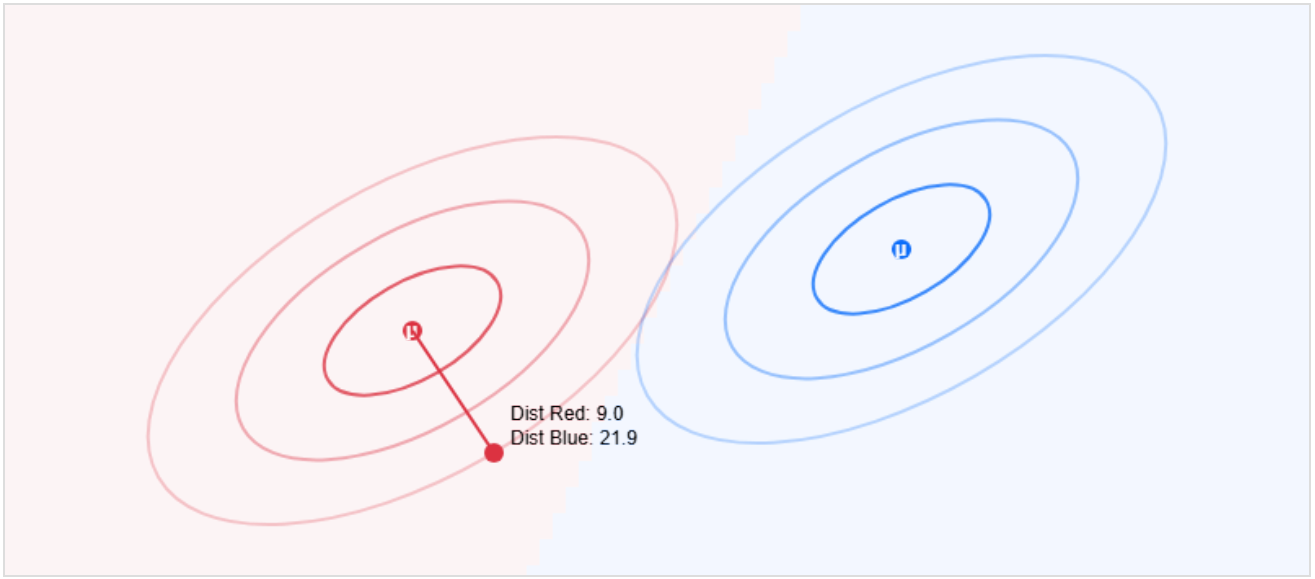
We can re-write this as:

$$\mathbf{w}^T \mathbf{x} + b \underset{1}{\overset{0}{\lessgtr}} 0 \quad \text{linear classifier}$$

Visualizing the Gaussian Assumption

Move your mouse over the canvas. The background color shows the decision regions derived from the two Gaussian distributions.

Notice how the shared oval shape (Covariance Σ) affects the decision boundary line.



4. Python Implementation

Using `LinearDiscriminantAnalysis` from scikit-learn.

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification
import matplotlib.pyplot as plt
import numpy as np

def lda_demo():
    """
    LDA demo for binary classification.
    """
    # 1. Generate Binary Data
    X, y = make_classification(n_features=2, n_redundant=0,
                               n_informative=2,
n_clusters_per_class=1,
                               class_sep=2.0, random_state=42)

    print(f"X Shape: {X.shape}")
    print(f"y Shape: {y.shape}")

    # 2. Split Data
    X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3)

    # 3. Apply LDA
    lda = LinearDiscriminantAnalysis()
    lda.fit(X_train, y_train)

    # 4. Predict & Score
    acc = lda.score(X_test, y_test)
    print(f"LDA Accuracy: {acc:.2f}")

    # 5. Plot Decision Boundary
    # LDA draws a line that separates the two classes.
    print("Coefficients:", lda.coef_)
    print("Intercept:", lda.intercept_)

    # Create meshgrid for visualization
```

```
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
                         np.arange(y_min, y_max, 0.02))

    # Predict for each point in meshgrid
    Z = lda.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # Plot
    plt.figure(figsize=(8, 6))
    plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolor='k',
cmap=plt.cm.coolwarm)
    plt.title('LDA Decision Boundary')
    plt.xlabel('Feature 1')
    plt.ylabel('Feature 2')
    plt.show()

## Calling the function
lda_demo()
```